



نام درس: شبکه های کامپیوتری  
نویسنده: محمد امانلو - ۸۱۰۱۰۰۰۸۴



خلاصه شماره  
۲۰

## خلاصه‌ی فصل 4 جلسه‌ی 4 و فصل 5 جلسات 1 و 2

در این مجموعه اسلایدها اول درباره‌ی گذار از IPv4 به IPv6 و مفهوم tunneling صحبت می‌شود و بعد کم‌کم بحث به سمت «صفحه‌ی داده» و «صفحه‌ی کنترل» در لایه‌ی شبکه، ایده‌ی generalized forwarding و معماری SDN و OpenFlow می‌رود و در نهایت به بحث middleboxها و اصول معماری اینترنت و استدلال end-to-end می‌رسد. بعد، فصل 5 وارد دنیای الگوریتم‌های مسیریابی و صفحه‌ی کنترل می‌شود؛ هم مدل سنتی «هر روتر برای خودش» را توضیح می‌دهد و هم مدل «کنترل متمرکز منطقی»، و سپس دو خانواده‌ی اصلی الگوریتم‌های مسیریابی یعنی link state و distance vector را با جزئیات، همراه با الگوریتم‌های Dijkstra و Bellman-Ford، مثال‌ها و مشکلاتی مثل «count to infinity» تشریح می‌کند.

در بخش گذار از IPv4 به IPv6 تأکید می‌شود که عملاً امکان ندارد همه‌ی روترها در یک روز و یک لحظه ارتقا داده شوند؛ چیزی به اسم flag day واقع‌بینانه نیست. بنابراین مدت طولانی شبکه باید در وضعیتی کار کند که بعضی روترها هنوز فقط IPv4 را می‌فهمند و بعضی‌ها IPv6 را هم پشتیبانی می‌کنند. برای این که بسته‌های IPv6 بتوانند از روی ناحیه‌هایی که فقط IPv4 دارند عبور کنند، از تکنیکی به نام tunneling استفاده می‌شود؛ یعنی یک دیتاگرام IPv6 را مثل یک «محموله» داخل یک دیتاگرام IPv4 می‌گذاریم، به‌طوری که روترهای میانی فقط هدر IPv4 را می‌بینند و آن را مثل یک بسته‌ی معمولی IPv4 جابه‌جا می‌کنند و روتر مقصد تونل دوباره هدر IPv6 را بیرون می‌کشد. روی اسلایدهای تونل (دید منطقی و فیزیکی) نشان داده می‌شود که بین دو روتر IPv6 در دو سر تونل، یک «مسیر مجازی» روی زیرساخت IPv4 ساخته می‌شود و در مسیر، آدرس‌های مبدأ و مقصد در لایه‌ی IPv4 با آنچه در هدر IPv6 است فرق دارد، چون در واقع «بسته داخل بسته» داریم. این ایده‌ی تونل‌زدن بعدها در 4G/5G و خیلی زمینه‌های دیگر هم به‌شدت استفاده شده است.

اسلایدهای بعدی آمار تقریبی از پذیرش IPv6 می‌دهند؛ مثلاً این که درصد قابل توجهی از کاربران Google از طریق IPv6 وصل می‌شوند و بخش قابل توجهی از دامنه‌های دولتی US هم IPv6 را پشتیبانی می‌کنند، ولی با این حال، فرآیند پذیرش خیلی طولانی بوده و بیش از ۲۵ سال است که این مهاجرت ادامه دارد، در حالی که در همین بازه‌ی زمانی، جهان نرم‌افزار و کاربردها دگرگونی‌های عظیمی مثل WWW، شبکه‌های اجتماعی، استریم ویدئو، بازی‌های

آنلاین و telepresence را تجربه کرده است. این تضاد برای این است که زیرساخت لایه‌ی شبکه چیزی نیست که به‌سادگی و سرعت بتوان آن را عوض کرد.

بعد، نقشه‌ی راهی برای «صفحه‌ی داده» در لایه‌ی شبکه داده می‌شود و گفته می‌شود که در کنار خود IP و نحوه‌ی ساخت و هدر دیتاگرام و آدرس‌دهی و مفاهیمی مثل NAT و IPv6، یک موضوع مهم دیگر هم وجود دارد و آن ایده‌ی generalized forwarding است. در این نگاه، هر روتر یک flow table دارد که توسط یک کنترلر مسیریابی متمرکز منطقی محاسبه و توزیع می‌شود؛ یعنی به جای این که هر روتر فقط روی آدرس مقصد IP تصمیم بگیرد، می‌تواند روی ترکیب‌های مختلفی از فیلدهای هدر تصمیم بگیرد. در مدل OpenFlow، هر سطر جدول جریان شامل یک بخش match است که روی فیلدهای هدر (آدرس IP، آدرس MAC، شماره پورت TCP، نوع Ethernet، VLAN ID و غیره) الگوی خاصی را می‌سنجد، یک بخش action که می‌گوید با بسته‌ای که این الگو را دارد چه کار کنیم (مثلاً دراپ، فوروارد روی پورت خاص، تغییر بعضی فیلدها یا ارسال به کنترلر) و بخش stats که آمار تعداد بسته و تعداد بایت‌های این جریان را نگه می‌دارد. در مثال اسلایدها، با استفاده از همین جدول جریان به راحتی می‌توان رفتارهایی مثل فوروارد کردن بر اساس آدرس مقصد IP، پیاده‌سازی یک firewall ساده که مثلاً همه‌ی ترافیک به پورت 22 را دراپ می‌کند، یا بلاک کردن ترافیک از یک host خاص را فقط با اضافه کردن چند قاعده‌ی match+action انجام داد. نکته‌ی مهم این است که همین انتزاع match+action دستگاه‌های مختلف شبکه را زیر یک چتر مشترک می‌آورد. یک router سنتی را می‌توان به‌صورت دستگاهی دید که روی «طولانی‌ترین پیشوند مقصد IP» match می‌کند و اکشنش فوروارد کردن روی یک لینک است؛ یک switch لایه‌ی دو روی آدرس MAC مقصد match می‌کند و عمل فوروارد یا flood را انجام می‌دهد؛ یک firewall روی آدرس‌های IP و شماره پورت‌های TCP/UDP match می‌کند و اکشنش permit یا deny است؛ و یک NAT روی آدرس و پورت IP match کرده و اکشنش بازنویسی آدرس و پورت است. به این ترتیب، OpenFlow و SDN یک زبان مشترک برای بیان رفتار داده‌مسیر برای انواع تجهیزات شبکه فراهم می‌کنند.

این انتزاع عمومی، زمینه را برای ظهور middleboxها فراهم کرده است؛ یعنی جعبه‌های میانی‌ای که کارهایی فراتر از «روتر IP ساده» روی مسیر انجام می‌دهند. در RFC 3234، middlebox به‌عنوان هر جعبه‌ای تعریف شده که روی مسیر بین host مبدأ و مقصد، کاری غیر از وظایف استاندارد یک روتر IP انجام دهد؛ مثل NAT، firewall، load balancer، IDS، cache و غیره. با وجود SDN، NFV و سخت‌افزارهای white-box که هم شبکه و هم محاسبه و هم ذخیره‌سازی را قابل برنامه‌ریزی می‌کنند، این middleboxها همه‌جا پیدا می‌شوند؛ از شبکه‌های خانگی و سلولی تا شبکه‌های سازمانی، ISPها و datacenterها. اسلاید «Middleboxes everywhere» دقیقاً نشان می‌دهد که در هر نوع شبکه‌ای چه نوع middleboxهایی معمولاً وجود دارند.

در ادامه، اسلاید «ساعت‌شنی IP» نشان می‌دهد که اینترنت یک «کمر باریک» دارد: در وسط، فقط یک پروتکل لایه‌ی شبکه یعنی IP وجود دارد که همه‌ی دستگاه‌های متصل به اینترنت باید آن را پیاده‌سازی کنند؛ اما بالا و پایین

آن، یعنی در لایه‌ی لینک و فیزیکی و در لایه‌ی حمل و کاربرد، تنوع زیادی از پروتکل‌ها مثل WiFi، Ethernet، Bluetooth، TCP، UDP، HTTP، SMTP، QUIC، RTP و غیره وجود دارد. نسخه‌ی «میان‌سالی» همین شکل، اضافه شدن middleboxها به داخل شبکه را به صورت «چربی‌های اضافه» اطراف کمر باریک نشان می‌دهد. بعد، اصول معماری اینترنت طبق RFC 1958 مرور می‌شود: هدف اصلی «وصل شدن» است، ابزار این هدف پروتکل IP است و هوش و پیچیدگی سیستم بیشتر در انتهای شبکه (یعنی end hostها) قرار دارد تا در خود شبکه. این نگاه با «استدلال انتها به انتها» یا end-to-end argument تقویت می‌شود؛ این استدلال می‌گوید هر تابعی که برای پیاده‌سازی درستش نیاز به آگاهی از وضعیت برنامه‌های دو سر ارتباط است (مثل تضمین reliability) عملاً فقط در خود end hostها می‌تواند «کامل و درست» پیاده شود و اگر شبکه هم نسخه‌ای از آن را پیاده کند فقط نقش بهبود کارایی را دارد. در نهایت، اسلاید «هوش کجاست؟» مقایسه می‌کند: در شبکه‌های تلفن سنتی قرن بیستم، هوش و محاسبه در سوئیچ‌ها بود، در اینترنت قبل از ۲۰۰۰ هوش بیشتر در لبه بود و بعد از ۲۰۰۰ با ظهور cloud و سرویس‌های عظیم، باز هم بخش زیادی از هوش در لبه (سرورها و دستگاه‌های انتهایی) است، نه در core شبکه.

فصل 5 با معرفی «صفحه‌ی کنترل» در لایه‌ی شبکه شروع می‌شود. دو وظیفه‌ی اصلی لایه‌ی شبکه یادآوری می‌شود: forwarding یعنی فرستادن هر بسته از ورودی روتر به خروجی مناسب، که در صفحه‌ی داده انجام می‌شود، و routing یعنی پیدا کردن مسیر از مبدأ تا مقصد و ساخت جدول‌های فورواردینگ، که در صفحه‌ی کنترل انجام می‌شود. دو رویکرد کلی برای طراحی صفحه‌ی کنترل مطرح می‌شود: در مدل per-router control plane، هر روتر برای خودش یک جزء الگوریتم مسیریابی دارد که با هم‌تایان روی روترهای دیگر پیام رد و بدل می‌کند تا در نهایت هر روتر جدول فورواردینگ خودش را بسازد؛ در مدل logically centralized control plane، یک کنترلر متمایز (معمولاً روی یک سرور از راه دور) با control agentهایی که روی روترها هستند صحبت می‌کند و جدول‌های فورواردینگ آن‌ها را محاسبه و نصب می‌کند؛ این همان معماری SDN است که در فصل قبل با OpenFlow برای صفحه‌ی داده توضیح داده شد.

قبل از ورود به جزئیات الگوریتم‌ها، چند نوع مسیریابی معرفی می‌شود. unicast زمانی است که یک مبدأ به دقیقاً یک مقصد بسته می‌فرستد؛ broadcast وقتی است که یک گره به همه‌ی گره‌ها در شبکه بسته می‌فرستد؛ multicast وقتی است که به یک زیرمجموعه‌ی خاص از گره‌ها فرستاده می‌شود؛ و anycast حالتی است که یک مقصد منطقی داریم ولی بسته به نزدیک‌ترین یا مناسب‌ترین عضو آن گروه تحویل داده می‌شود. شکل اسلاید انواع این سناریوها را به صورت گره‌های قرمز و سبز و زرد نشان می‌دهد. همچنین ارتباط بین مسیریابی و فورواردینگ تشریح می‌شود: الگوریتم مسیریابی مسیر سرتاسری بین مبدأ و مقصد را تعیین می‌کند، در حالی که جدول فورواردینگ در هر روتر فقط تصمیم محلی می‌گیرد که اگر بسته‌ای با یک آدرس مقصد مشخص رسید، از کدام خروجی ارسال شود؛ بنابراین، مسیریابی ورودی الگوریتمی است که آن جدول‌های محلی را پر می‌کند.

برای مدل‌سازی شبکه از انتزاع گراف استفاده می‌شود. شبکه به صورت گراف  $G = (N, E)$  نمایش داده می‌شود که

در آن  $N$  مجموعه ی روترها و  $E$  مجموعه ی لینک ها است. هر یال  $(x, x')$  یک هزینه ی  $c(x, x')$  دارد که می تواند همیشه 1 باشد یا مثلاً عکس پهنای باند یا تابعی از ازدحام. هزینه ی یک مسیر  $u(x_1, x_2, \dots, x_p)$  و  $z$ ، چیست؛ الگوریتم مسیریابی دقیقاً الگوریتمی است که این مسیر کم هزینه را پیدا می کند. سپس الگوریتم ها براساس دو معیار دسته بندی می شوند: از نظر «جهانی یا توزیع شده بودن اطلاعات»، الگوریتم های link state آنهایی اند که هر روتر دید کاملی از توپولوژی و هزینه ی همه ی لینک ها دارد (با link state broadcast)، در حالی که الگوریتم های distance vector فقط هزینه ی لینک به همسایه های مستقیم را می دانند و بقیه ی اطلاعات را به صورت تدریجی و با مبادله ی بردار فاصله با همسایه ها به دست می آورند؛ از نظر «ایستا یا پویا بودن»، الگوریتم های استاتیک در زمان کوتاه تغییر نمی کنند، اما در الگوریتم های پویا مسیرها در پاسخ به تغییر هزینه ی لینک ها یا به صورت دوره ای به روز می شوند.

در بخش link state، الگوریتم Dijkstra معرفی می شود که یک گره (مثلاً گره ی source) با داشتن توپولوژی کامل شبکه و هزینه ی همه ی لینک ها، کم هزینه ترین مسیر خود به همه ی مقصدها را محاسبه می کند. هر گره پس از اجرای Dijkstra می تواند جدول فورواردینگ خودش را بسازد. برای این کار از نمادهایی مثل  $c(x, y)$  برای هزینه ی لینک بین  $x$  و  $y$ ،  $D(v)$  برای بهترین هزینه ی فعلی از مبدأ به  $v$ ،  $p(v)$  برای پیش کسوت روی مسیر فعلی و  $N'$  برای مجموعه ی گره هایی که مسیرشان قطعی شده استفاده می شود. الگوریتم به طور خلاصه این طور است که ابتدا مبدأ را داخل  $N'$  می گذارد و هزینه ی همسایه های مستقیم را مقداردهی می کند؛ سپس در هر گام، گره ای را که کمترین هزینه ی  $D(w)$  را دارد و هنوز در  $N'$  نیست انتخاب کرده، آن را به  $N'$  اضافه می کند و هزینه های همسایه هایش را با رابطه ی

$$D(v) \leftarrow \min(D(v), D(w) + c(w, v))$$

به روز می کند. پس از  $n$  تکرار، مسیرهای کم هزینه به همه ی گره ها پیدا شده اند. مثال های اسلاید نشان می دهد که چگونه با استفاده از جدول های میانی می توان در نهایت «درخت مسیرهای کم هزینه» و جدول فورواردینگ برای گره مبدأ را به دست آورد. همچنین درباره ی پیچیدگی الگوریتم گفته می شود که در نسخه ی ساده حدود  $O(n^2)$  (oscillation  $O(n \log n)$ ) شود؛ چون هر بار با تغییر مسیرها، ترافیک و در نتیجه هزینه ها عوض می شوند و الگوریتم دوباره مسیر جدیدی پیدا می کند و این چرخه ممکن است تکرار شود. در بخش distance vector، الگوریتمی مبتنی بر معادله ی Bellman-Ford معرفی می شود. ابتدا خصوصیات کلی گفته می شود: این الگوریتم کاملاً توزیع شده است، چون هر گره فقط با همسایه های مستقیم خود صحبت می کند؛ تکراری و خودپایان است، یعنی تا زمانی که لازم باشد گره ها به تبادل اطلاعات ادامه می دهند و دیگر نیازی به سیگنال بیرونی برای توقف نیست؛ و ناهمگام است، یعنی لزوماً همه ی گره ها در یک زمان مشخص به روز نمی شوند و الگوریتم با پیام های نامنظم هم کار می کند. معادله ی Bellman-Ford می گوید هزینه ی کم هزینه ترین مسیر از  $x$  به  $y$  (یعنی  $d_x(y)$ ) برابر است با

$$d_x(y) = \min_v (c(x, v) + d_v(y))$$

که در آن  $v$  روی همه ی همسایه های  $x$  می چرخد. به زبان ساده، گره  $x$  برای رسیدن به  $y$  می تواند از هر همسایه ی  $v$

عبور کند؛ هزینه‌ی رفتن از  $x$  به  $v$  را می‌داند و اگر بداند  $v$  چقدر تا  $y$  فاصله دارد، می‌تواند جمع این دو را حساب کند؛ کمترین این جمع‌ها بهترین مسیر است.

در الگوریتم distance vector هر گره  $x$  یک بردار فاصله  $D_x$  نگه می‌دارد که شامل تخمین‌های فعلی‌اش از هزینه‌ی رسیدن به همه‌ی مقصدهاست، یعنی  $D_x(y)$  تقریب  $d_x(y)$  است. علاوه بر این،  $x$  هزینه‌ی لینک تا هر همسایه‌ی مستقیم  $c(x,v)$  را می‌داند و نسخه‌ی فعلی بردار فاصله‌ی همسایه‌هایش، یعنی  $D_v$  برای هر همسایه‌ی  $v$ ، را در حافظه نگه می‌دارد. هر از گاهی، هر گره بردار فاصله‌ی فعلی خودش را برای همسایه‌ها می‌فرستد. وقتی  $x$  بردار فاصله‌ی جدید  $v$  را دریافت کرد، برای هر مقصد  $y$  رابطه‌ی Bellman-Ford را اعمال می‌کند و اگر مقدار جدید  $D_x(y)$  از قبلی کمتر شد، بردار فاصله‌ی خودش را به‌روزرسانی می‌کند. اگر این بردار تغییر کند،  $x$  دوباره آن را برای همسایه‌ها می‌فرستد. تحت شرایط معمول، این فرآیند به‌صورت تدریجی همگرا می‌شود و بردارهای فاصله‌ی گره‌ها به مقادیر واقعی کم‌هزینه‌ترین مسیرها می‌رسند.

برای فهم بهتر رفتار الگوریتم، مفهوم distance table معرفی می‌شود که در هر گره یک جدول دو بعدی دارد: سطرها مقصدهای ممکن هستند و ستون‌ها همسایه‌های مستقیم. هر خانه‌ی جدول مثلاً  $d_E(A, B)$  یعنی هزینه‌ی رسیدن از گره  $E$  به مقصد  $A$  از طریق همسایه‌ی  $B$ . با اعمال معادله‌ی Bellman-Ford روی این جدول، مقدار هر خانه از روی هزینه‌ی لینک تا همسایه و کمترین هزینه‌ی همسایه تا مقصد به‌دست می‌آید. با نگاه کردن به هر سطر و انتخاب کوچک‌ترین مقدار، هم هزینه‌ی کم‌هزینه‌ترین مسیر به آن مقصد تعیین می‌شود و هم این‌که از کدام همسایه باید استفاده شود؛ این می‌شود جدول فورواردینگ گره. اسلایدها یک مثال کامل برای گره‌ی  $E$  نشان می‌دهند که چگونه از جدول فاصله، جدول مسیریابی بیرون کشیده می‌شود.

رفتار الگوریتم در برابر تغییر هزینه‌ی لینک‌ها هم بررسی می‌شود. وقتی هزینه‌ی یک لینک کم می‌شود (مثلاً لینک بین  $x$  و  $y$  ارزان‌تر می‌شود)، «خبر خوب» سریع پخش می‌شود؛ چون گره‌ای که تغییر محلی را می‌بیند بلافاصله بردار فاصله‌ی جدیدش را می‌فرستد، همسایه‌ها آن را بهبود می‌دهند و به نوبه‌ی خود پیام می‌فرستند و معمولاً طی چند مرحله کم‌شمار، الگوریتم به وضعیت پایدار جدید می‌رسد. اما وقتی هزینه‌ی لینکی زیاد می‌شود یا لینکی قطع می‌شود، مشکل معروف count to infinity ظاهر می‌شود: چون گره‌ها فقط بردار فاصله‌شان را می‌فرستند و نه ساختار کامل مسیر را، ممکن است برای مدتی طولانی تصور کنند که هنوز مسیر ارزان‌تری از طریق گره‌های دیگر وجود دارد و به تدریج مقدار هزینه را بالا ببرند؛ در این مدت، «خبر بد» خیلی کند پخش می‌شود. مثال اسلاید با سه گره‌ی  $X, Y, Z$  نشان می‌دهد که وقتی هزینه‌ی لینک  $X-Y$  زیاد می‌شود، گره‌ها بارها و بارها هزینه‌ی مسیر به  $X$  را از طریق یکدیگر محاسبه می‌کنند و هر بار کمی بیشتر می‌شود تا بالاخره به مقدار درست برسد؛ این همان «کند بودن انتشار خبر بد» است.

برای کاهش این مشکل، چند ترفند عملی معرفی می‌شود. یکی split horizon است که می‌گوید هر گره وقتی به یک همسایه، اطلاعات مسیریابی می‌فرستد، مسیرهایی را که از خود آن همسایه یاد گرفته، دوباره به خودش برنگرداند؛ مثلاً

اگر B مسیر (E,2) را از A یاد گرفته، هنگام ارسال آپدیت به A دیگر این ورودی را در بسته‌ی مسیریابی نمی‌گذارد. نسخه‌ی قوی‌تر split horizon with poison reverse است؛ در این جا گره نه‌تنها ورودی را حذف نمی‌کند، بلکه آن را با هزینه‌ی بی‌نهایت برای همان همسایه می‌فرستد تا به او صراحتاً بگوید «من از تو به این مقصد نمی‌رسم، پس هرگز من را برای این مقصد به عنوان مسیر بعدی در نظر نگیر». حتی با این روش‌ها هم هنوز حلقه‌های بزرگ‌تر از دو گره می‌توانند مشکل‌ساز شوند و گاهی از روش‌هایی مثل triggered update یا محدود کردن حداکثر مقدار هزینه (مثلاً عددی مثل ۱۶ در RIP) استفاده می‌شود.

در پایان، اسلایدها یک مقایسه‌ی جمع‌بندی بین الگوریتم‌های link state و distance vector ارائه می‌کنند. از نظر پیچیدگی پیام، در link state هر گره مجبور است وضعیت لینک‌هایش را برای همه منتشر کند و در شبکه‌ای با n گره و E لینک، در حدود  $O(nE)$  پیام ب

مراجع